SPFx Extensions, not web parts

Code that lives next to the page rather than inside its own canvas. Four official flavours, one set of deployment rules.

A Application Customizer

`top / bottom placeholders`

- JSX injected into named page placeholders
- Header, footer, top of every modern page
- Service scope, no DOM chrome of its own
- Banners, nav links, telemetry pings

F Field Customizer

`cell render, by field`

- Replace the cell view with custom JSX
- Status pills, progress bars, profile chips
- Bound to a specific column internal name
- Read-only by default — open a panel to edit

L List View Command Set

`toolbar, context menu`

- Adds buttons to the list toolbar
- Items surface in row context menu
- Single / multi selection aware
- Approve, archive, open side panels

M Form Customizer

`new, replaces list forms`

- New form, edit form, view form per list
- React-based, page owns its own layout
- Replaces the legacy InfoPath surface
- Best when rules get complex

Extensions vs web parts — they don't compete

Web part

- Owns a chunk of DOM a user can place and resize
- Stored in the page, edited in the canvas
- Users pick them up from the toolbox
- Visible only on the page it sits in

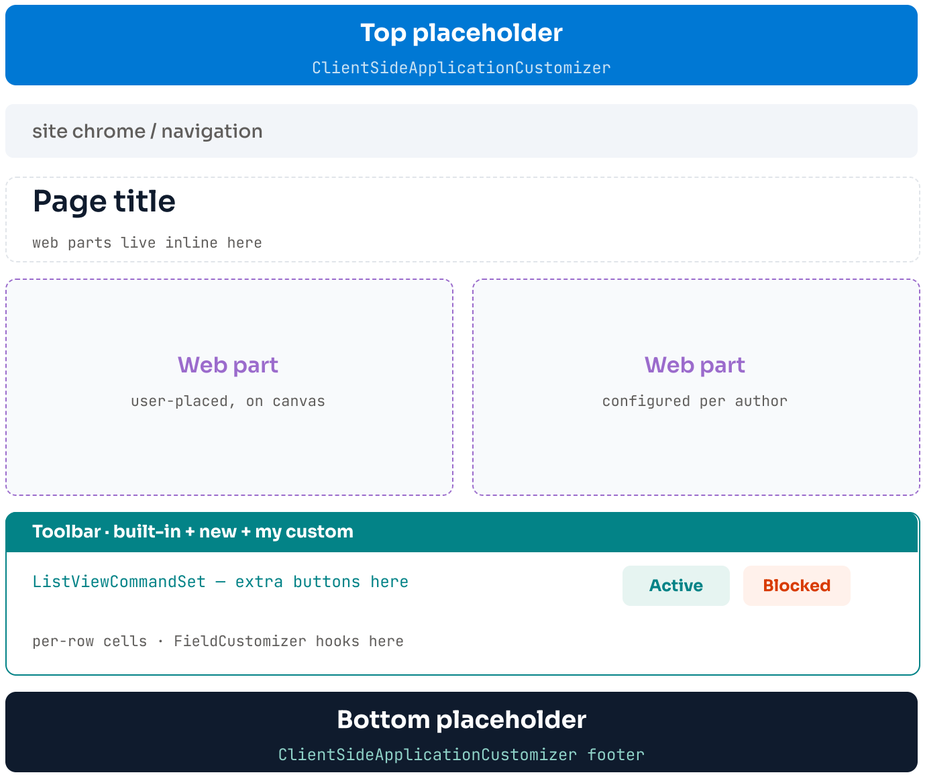
Extension

- Hides inside named page slots the site already exposes
- Activated tenant-wide or per site — not by the author
- Reaches every matching page without anyone dragging it in
- Often paired with a web part for the user-facing surface

Where each extension hooks the page

A modern page already exposes 6 named slots — knowing each one is half the design.

Anatomy of a modern page



Six slots, four hooks

1. **Top placeholder**
   `ApplicationCustomizer`
2. **Page canvas / web parts**
   `no extension hook — this is web part territory`
3. **List toolbar & context menu**
   `ListViewCommandSet`
4. **Per-cell render**
   `FieldCustomizer, by column internal name`
5. **New / Edit / View form**
   `FormCustomizer, per list`
6. **Bottom placeholder**
   `ApplicationCustomizer footer`

Two slots are still off-limits to extensions: search box and side nav. The rest is yours the moment you ship a package.

Application Customizer: placeholders

Inject JSX into named placeholders, run in service scope, register the placeholder names in the manifest.

What you get to use

Named placeholders

`PageHeader` · `PageFooter` · `PageTop` · your own via `PlaceholderName=x`

BaseExtensionContext

`this.context.pageContext`, `this.context.spHttpClient`, `this.context.serviceScope`

onInit lifecycle

One call per page load — mount, optionally react to listChanged events later

- Binds once per page; no canvas, no resize handles
- Deployed tenant-wide, not added by an author
- Renders in DOM order — last write to placeholder wins
- Does not own the page; defer to existing chrome
- Keep it small — tens of KB, runs on every page view

onInit + render pattern

// src/extensions/footer/FooterApplicationCustomizer.ts

```ts
import { override } from '@microsoft/decorators';
import { Log } from '@microsoft/sp-core-library';

const LOG_SOURCE = 'FooterApplicationCustomizer';

export interface IFooterProps {
  message: string;
}

export default class FooterApplicationCustomizer
  extends BaseApplicationCustomizer<IFooterProps> {

  @override
  public async onInit(): Promise<void> {
    Log.info(LOG_SOURCE, `initialised on ${this.context.pageContext.web.title}`);
    const top = this.context.placeholderProvider.tryCreateContent(
      PlaceholderName.Top,
      { onDispose: this._onDispose }
    );
    if (top) {
      top.domElement.innerHTML = `<div class="ms-bgColor-neutralLighter">
        ${this.properties.message}</div>`;
    }
  }

  private _onDispose(): void { /* teardown */ }
}
```

Real use: a tenant notice banner

Banner text read from a config list. Mark-as-read handles local storage; admins push updates without a re-deploy.

Scenario

Legal has rolled out a new GDPR retention rule. They want a banner at the top of every modern page until users acknowledge it.

A config list — one row per banner

```
/sites/legal/Banners

Title       GDPR retention policy update
Body        We updated our retention...
            (truncated)
```

- Fetch current active banner from the list on `onInit`
- Stamp *localStorage* so the banner stays dismissed per device
- Re-run with a fresh row whenever legal changes — no rebuild, no redeploy

BannerApplicationCustomizer.tsx

```
// reads active banner from /sites/legal/Banners

const banner = await sp.web.lists
  .getByTitle("Banners")
  .items.filter(`Active eq 1`)
  .top(1)();

const dismissed = localStorage.getItem(`banner-${row.Id}`);

const root = document.createElement("div");
root.className = "bn-banner bn-banner--info";
root.innerHTML =
  `<span>${row.Title}</span><button>Got it</button>`;
root.querySelector("button")!.onclick = () => {
  localStorage.setItem(`banner-${row.Id}`, "1");
  root.remove();
};

placeholder.domElement.appendChild(root);
```

contoso.sharepoint.com

GDPR retention policy update · please review Got it

Banner respects localStorage so re-deploys do not nag acknowledged users; index the Title column for fast lookups.

Field Customizer: paint cells

One extension serves every row of one column. Replace the raw value with your own JSX — keep editing on the form.

PriorityField Customizer — coloured badges

```ts
// src/extensions/priority/PriorityFieldCustomizer.ts

import * as React from "react";
import { Log } from "@microsoft/sp-core-library";

const palette = {
  High: "#D83B01", Med: "#FFB17A", Low: "#038387"
} as const;

export default class PriorityField
  extends BaseFieldCustomizer<unknown> {

  @override
  public onRenderCell(event: FieldCustomizerCellEventParameters): void {
    const value: string = event.fieldValue ?? "";
    const color = (palette as any)[value] ?? "#605E5C";
    event.domElement.innerHTML =
      `<span class="pf-badge" style="background:${color}22;color:${color
      + `<span class="pf-dot" style="background:${color}"/>${value}</span>`;
  }

  @override
  public onDisposeCell(event: FieldCustomizerCellEventParameters): void {
    event.domElement.innerHTML = "";
  }
}
```

Manifest registration

```ts
// serve.json · associations
{
  "internalName": "Priority",
  "id": "PriorityField",
  "title": "Priority badge"
}
```

Rendered output

High Med Low

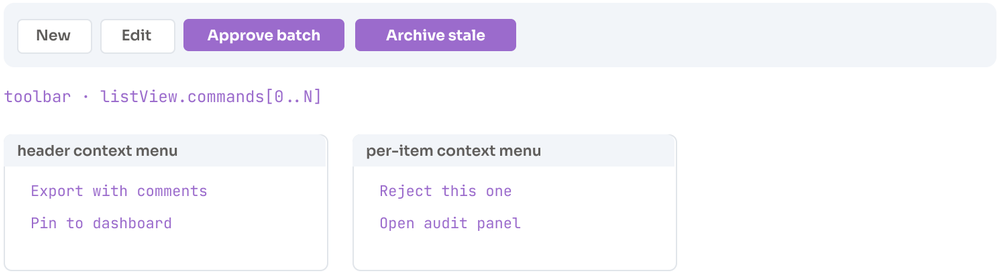
Rendering is read-only by default.
Use `OnBeforeCellRender` if you need to fetch extra data per row; debounce Graph calls inside that hook.

Cells are not editable here — users edit via the form. The customizer is a view layer, not a binding; keep `fieldValue` authoritative and ignore mutated DOM.

List View Command Set: toolbar

Inject commands into the list command surface. Show, hide, disable per row — or per user.

Where commands show up



Each command lives in a CommandSet. The same extension can publish several command sets; visibility / checked state are runtime decisions.

onExecute with permission gate

```ts
@override
public onExecute(event: ListViewCommandSetExecuteEventParameters): void {
  const selected = event.selectedRows;
  switch (event.itemId) {
    case "APPROVE_BATCH":
      if (!this._canApprove(selected)) return;
      this._approve(selected);
      break;
    case "REJECT_ITEM":
      this._reject(selected[0]);
      break;
  }
  event;
}

private _canApprove(rows: readonly any[]): boolean {
  const groups = this.context.pageContext.legacyPageContext.userGroups;
  const ok = groups.some(g => /Contoso.Approvers|M365 Owners/.test(g));
  return ok && rows.length > 0;
}

private async _approve(rows: readonly any[]) {
  await sp.web.lists.getByTitle(rows[0].listName)
    .items.filter(`Status eq 'Pending'`)
    ...
}
```

Tip · toggle `command.hidden / command.disabled` in *onListViewUpdatedStateChanged*

Re-evaluate on every selection change — return a new array of commands; the framework reconciles.

Custom forms: three options

The list form gets replaced, customised, or sidestepped. Pick the smallest tool that does the job.

Power Apps custom form

canvas app · lists, library

- Configured on a list column — SharePoint calls into a canvas app
- Pure Power Fx, no React, no build pipeline
- Edit, view, new each get a different form
- Ships with the list, no .sppkg to deploy

Choose when

Maker owns the form, rules stay readable, people who build SPFx are not needed.
Worries: Power Fx limits how far the UX can push.

Tip · Customise the SharePoint form control with `SharePointIntegration` — reads/writes the column you bound it to.

SPFx Form Customizer

React, code-owned, complex logic

- React page replaces new / edit / view form
- Full TypeScript, full SP REST, full Graph
- Deployed as an .sppkg through App Catalog
- Bears the same auth + ALM story as web parts

Choose when

Rules cross services, validation needs custom hooks, the UX must mirror a React design system.
Worries: code review, npm patching, owns ALM.

Tip · Hook list form props with `getFormItemsByID` for fast typed reads in FormContext.

Embedded Power Apps

canvas app, sit next to content

- A canvas app placed inside an SPFx web part
- Neither owns the form — coexists with default
- Powered by Power Apps SDK: `PowerAppsComponent`
- Helpful for read-heavy side panels

Choose when

Some flows stay Power Fx friendly, but you also need code around them. A read-mostly panel sits next to an existing form.

Tip · Pass `appId` & `environmentId` from property pane — keep one app per use case.

Power Automate: flow types

Three trigger patterns cover almost every SharePoint scenario. Pick by who acts, not by which is newest.

A — Automated

`event-triggered`

- When an item is *created / modified*
- When a file is *created / modified*
- Property filters keep noise out
- Retries are managed for you — 4 by default, then 30 days

I — Instant

`user-triggered`

- Manual trigger via Power Automate, button, or Power Apps
- Pass inputs from call sites, validate in the flow
- Best when humans decide to run something
- Often reaches SPFx via *Send HTTP request*

S — Scheduled

`clock-triggered`

- Recurrence — hourly, daily, custom expression
- Useful for nightly sync, retention sweeps
- Cheap and slow beats an expensive real-time flow
- Watch for per-flow run quotas

Send HTTP request — your bridge to REST and Graph

```
// Power Automate step body

Method: POST
Uri: _api/web/lists/getbytitle('Projects')/items
Headers:
  Accept: application/json;odata=verbose
  Content-Type: application/json;odata=verbose
Body:
  { "Title": "@{triggerOutputs()?['body/Title']}",
    "Status": "Approved", "Approver": "@{outputs('Who')?['mail']}" }

// works because the flow runs in user context — consent still scopes it
```

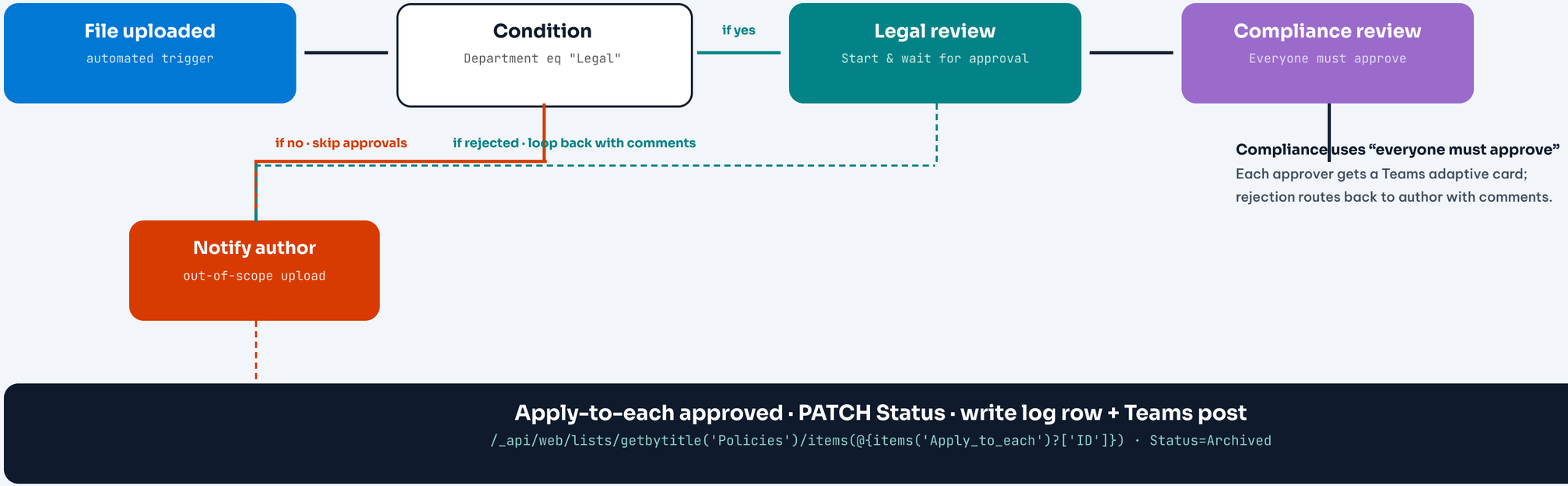
Approval shapes that ship

- *Approve / Reject — First to respond*
- *Approve / Reject — Everyone must approve*
- *Custom Responses* — state outcomes
- Each approval action carries its own email + adaptive card

Tip · chain parallel branches with `Apply-to-each` and conditions, not nested *Switch*.

Document approval chain

A library called *Policies* — uploads route through legal, then compliance, then archive. Three branches, one rule.



Compliance uses “everyone must approve”
Each approver gets a Teams adaptive card; rejection routes back to author with comments.

Decisions this flow makes

- Filters on *Department* so non-policy uploads skip review
- Two parallel chains keep waiting overlap short
- Rejections loop back to the author with comments

Anti-patterns to retire

- ✕ One mega-flow per scenario — split, share via child flows
- ✕ Hard-coded email recipients — use Entra groups
- ✕ Polling on the modified list — switch to *When modified*
- ✕ Silent HTTP — branch on *statusCode*

Azure Functions: when SPFx hits a wall

Offload the heavy lift. Run server-side jobs with managed identity and call Graph without storing secrets client-side.

Long-running work

Imports over 5 MB, batched migrations, nightly reports. The browser is not the place.

Secrets & certificates

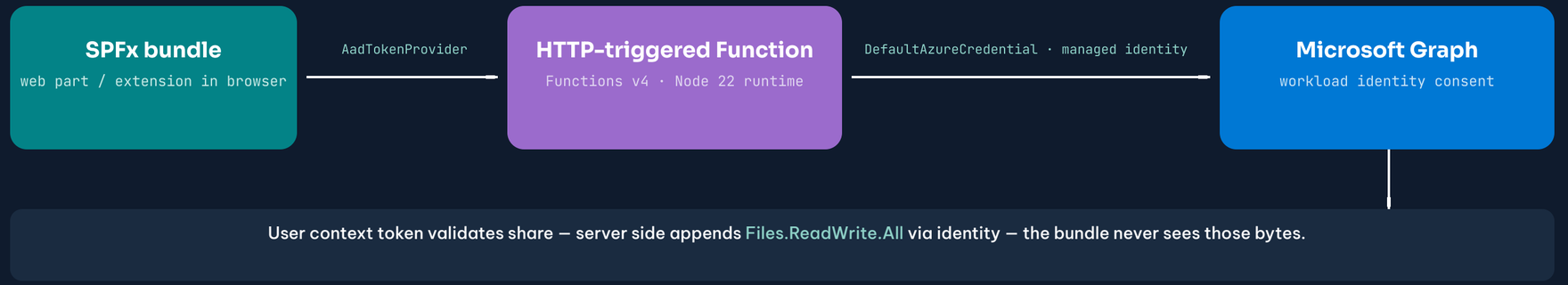
Anything that needs a server-held key — private connectors, premium Graph surfaces.

Cross-tenant calls

Partners, vendors, service-account calls — keep these off user tokens.

SPFx → Function → Graph — the loop

User browser · SPFx bundle · AadTokenProvider · HTTP trigger · managed identity



`DefaultAzureCredential` in Azure Functions v4 picks up the function app's system-assigned identity. No client secret, no rotation, no SP-leak risk in the bundle.

Long jobs with queues

HTTP-trigger returns 202 immediately, queue-trigger worker drains work — user never waits for the long path.

HTTP enqueue · Node 22

```ts
// src/functions/enqueueMigration.ts

import { app, HttpRequest, HttpResponseInit } from "@azure/functions";
import { QueueClient } from "@azure/storage-queue";

const queue = new QueueClient(process.env.MIG_QUEUE_URL!);

app.http("enqueue", {
  methods: ["POST"], authLevel: "anonymous",
  handler: async (req: HttpRequest) => {
    const jobId = crypto.randomUUID();
    await queue.sendMessage(Buffer.from(JSON.stringify({ jobId, ...req.body })));
    return { status: 202, jsonBody: { jobId } }
  }
});
```

Worker · queue trigger drains it

```ts
// src/functions/worker.ts

import { app } from "@azure/functions";
import { ClientSecretCredential } from "@azure/identity";

app.storageQueue("migrate", {
  connection: "AzureWebJobsStorage",
  handler: async (msg, ctx) => {
    const cred = new DefaultAzureCredential();
    const token = await cred.getToken("https://graph.microsoft.com/.default");
    await runMigration(msg, token!.token);
    ctx.log(`done ${msg.jobId}`);
  }
});
```

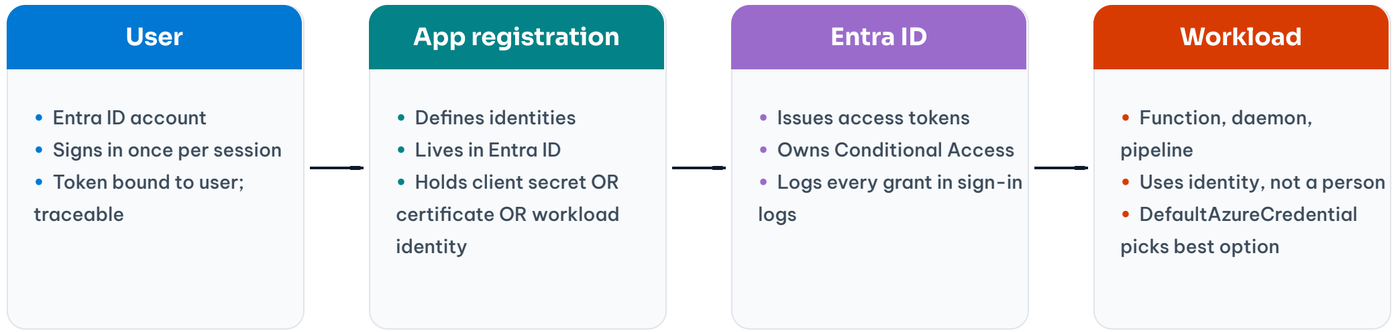
Why this pattern beats a tight loop

- Browser never holds long jobs; users get a jobId, hit a status endpoint, see progress
- Worker scales independently — Functions Premium plan consumes the queue in parallel
- Poison messages & dead-letter keep errors visible, not silently dropped
- Pair with Application Insights for end-to-end traces — one *operation_Id* across both

Microsoft Entra ID: tokens

The directory that issues every token. Pick the credential model that matches the risk of the work.

A token, four shoulders



token → Graph validates issuer, audience, scopes → works scoped to the request size you asked for

Picking the credential

01 User token SPFx, Power Apps

02 Managed identity Functions, Logic Apps

03 Certificate CI/CD, automated jobs

04 Client secret legacy fallback, low trust

Definitions worth memorising

Tenant ID · the directory your app lives in; often `/sites/…` URLs omit but tokens carry it.

Audience · what resource the token is for — graph.microsoft.com (Graph), sharepoint.com (SP REST).

Scope · what actions the token may perform — smallest set wins.

Claim · piece of info inside a token — preferred_username, groups, roles, oid.

Naming note

Azure AD was renamed Microsoft Entra ID in 2023. New code says Entra ID; old docs say Azure AD — same surface.

Use `login.microsoftonline.com` for v1 tokens, `/common` for multi-tenant flows.

Permissions & the consent flow

Ask for the smallest scope that works. Make consent visible. Honour revocation — it does happen.

Ask in `package-solution.json`

```
{
  "webApiPermissionRequests": [
    {
      "resource": "Microsoft Graph",
      "scope": "Sites.Selected"
    },
    {
      "resource": "Microsoft Graph",
      "scope": "User.ReadBasic.All"
    }
  ]
}
```

Sites.Selected — the safe default

- App can act only on sites the admin grants
- Per-site grant: *POST /sites/{id}/permissions* with *roles*
- *read*, *write*, or *owner*
- Revoke with *DELETE* on the same endpoint
- Avoid *Sites.ReadWrite.All* unless you really mean the whole tenant

User → admin consent → token → honour revocation



Admin consent can be done once with `Grant Admin Consent` in Entra, or scripted through *Mg cli*. Build a re-consent endpoint in your app in case a tenant admin resets — surface 401 with a clear link, do not silently swallow.

Delegated vs application permissions

Two ways for the same token. They answer different questions, audit differently, fail differently.

Dimension	Delegated	Application
Who acts	A signed-in user on behalf of the app	The app alone, no user present
Token	`access_as_user`	`client_credentials`
Effective rights	Union of (app scope) ∩ (user rights)	App's grants only — subject to admin
Audit signal	User in *oid* claim, all activity tied to person	App service principal; no user IP
Failure shape	403 if user role missing — ask re-consent	401 if grant revoked, 403 if scope missing
Typical use	SPFx, Power Apps, browse-time calls	Functions, daemons, scheduled jobs
Risk surface	Per-user liability; gate with caution	Higher blast radius; logs are vital

Sites.Selected — lives on the application side

Application-only scope. Admin grants per site; combined with delegated you can scope *both* sides. Use this instead of `Sites.ReadWrite.All` whenever the surface is small enough to enumerate.

Three ways to react to a list change

SharePoint webhooks, Graph subscriptions, and Remote Event Receivers compete on coverage, latency, and how much code you must own.

SharePoint webhooks

`/SP api/web/lists/{id}/subscriptions`

Reach
Per list — the webhook owns a single list's lifetime.

Latency
Near-real-time; 5 min back-stop if your endpoint stalls.

Lifetime
180 days default; renew or it silently dies.

Notification
Tells you *something changed* — call GetChanges by token to learn what.

Best for
Per-site automations, Azure Function listeners, MS-hosted handlers.

Graph subscriptions

`/v1.0/subscriptions`

Reach
Across M365 — chats, files, calendars, security alerts.

Latency
Same near-real-time; some resources batch in 1 minute windows.

Lifetime
Per resource type — some 4230 min, mail 4230, drive 4230.

Notification
Encrypted resourceData with full change payload — rich + lifecycle.

Best for
Cross-M365 reactions, Teams apps, mail flows.

Remote Event Receivers

`legacy · .NET, host on IIS or Azure`

Reach
Synchronous with the user action — before / after.

Latency
Inline with the operation — can cancel the action.

Lifetime
Tied to feature & receiver XML — tied to deploy.

Notification
Typed payloads via SPEventReceiverBase — cancel-friendly.

Best for
Cancel-before, custom validation, on-prem farms. Migrations to SP Online — phased out long-term.

Webhook lifecycle, then renew

Webhooks die silently at 180 days. Plan to renew or your handler will go quiet — no error, no signal.



Renew before expiration, or it ends

```
// Graph pattern: PATCH every 60 days

await graph
  .api(`/subscriptions/${subId}`)
  .update({
    expirationDateTime: new Date(Date.now() + 60 * 86400_000).toISOString()
  });

// SP REST: PATCH on the same id, only expiration
PUT /_api/web/lists('id')/subscriptions('id')
```

Failure modes to expect

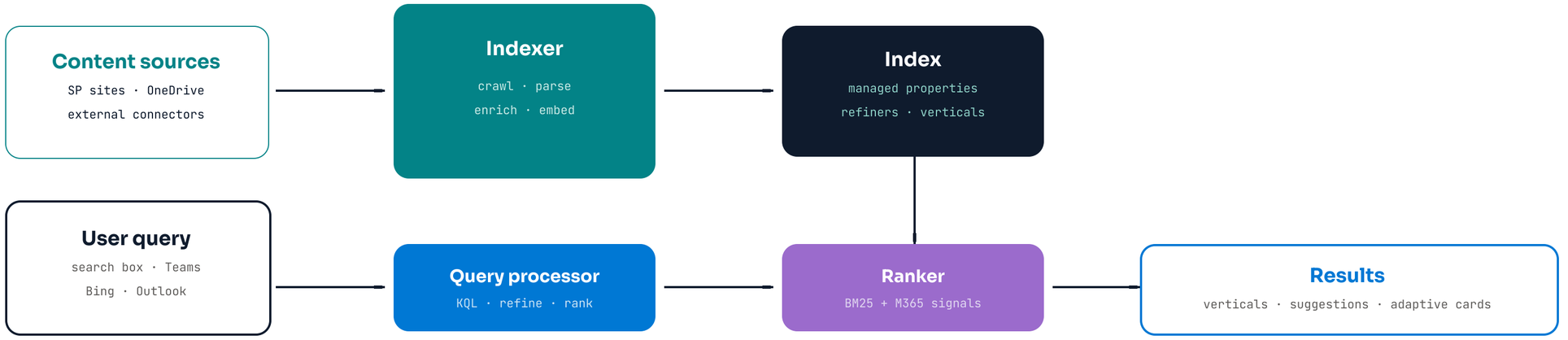
- Endpoint 500s — notification queued, retried for 4 hours
- Token expired mid-batch — restart from the last bookmark
- Subscription silently disabled — poll and re-enable
- List deleted — subscription vanishes, no notification
- Throttling returns 429 — skip ahead, retry later

Add a renewal timetrigger — renew 7 days early and save the subscription id table.

Search: index, query, rank

Microsoft Search rides on top of SharePoint index — same store, broader experience, Graph-friendly.

The pipeline



Four verticals out of the box

Files
Author · lastModifiedTime · FileType

Sites
SiteTitle · Department · URL

People
JobTitle · Department · Skills

Conversations
Outlook · Teams · Yammer

Indexing is asynchronous. Re-indexing happens automatically when a list schema changes — force a crawl only if a property maps the wrong way.

`Graph: /search/query · ?requests=[{entityTypes, query}] · SP REST: _api/search/postquery`

KQL & the Modern Search web parts

Write queries you can hand to a senior engineer and a junior — KQL is a tiny language with a few strong conventions.

KQL examples · four shapes you will reuse

simple filter

```
Author:"Pat Rowan" AND Status:"Active"
```

range & sort

```
DueDate ≥ 2026-01-01 AND PercentComplete < 100
  // ordered by SortableBehavior
```

managed property reuse

```
Path:"https://contoso.sharepoint.com/sites/HR/*"
  // using RefinableStringXX also works
```

group + count

```
Department:"Legal" AND IsDocument:true
  // aggregate via ?$select=Department,count()
```

Graph filter syntax is similar — same operators, plus $filter / $orderby / $select.

PnP Modern Search

- Search Results web part
- Search Box + filter web parts
- PnP Modern Search v4
- Layouts are Handlebars-style
- Configure KQL in property pane
- *Ships* via App Catalog like any SPFx

KQL gotchas

- Free-text terms and operators in the same query don't always combine
- Use *queryVariables* for tenant-flexible placeholders
- Map crawled properties first — new ones show up overnight

Performance: a working checklist

Ship a small bundle, hit one index per call, cache reads you can defend. Measure before declaring it fast.

B Bundle size

- Dynamic `import()` for charts, modals, settings panes
- One web part, one lazy chunk
- Treeshake the entire dep tree
- Stay under SPFx budget targets (aim initial < 500 KB)
- Heft instead of gulp for newer projects
- Read *bundle-analysis* after every change

D Data perf

- `.select(...)` columns you actually need
- Index columns you filter on
- Batch with PnPjs when > 5 ops
- Use *$expand=fields* only when you must
- Prefer Graph *delta* over polling
- Cap `.top` — render < 100 unless searched

C Caching

- PnPjs `DefaultCachingWith` behaviours on *spfi*
- Cache *me*, site metadata, taxonomy
- Skip cache for write operations
- Switch cache store: *session*, *local*, or *null*
- Always document invalidation rules
- TTL = smallest that still feels live

Tooling · bundle-analyzer, Network panel, Application Insights, `_api/web/HubSiteData`?

Capture *operation_Id* end to end. The browser shows milliseconds; Insights shows who paid them.

Lab: extension + flow + function

One tenant-wide banner, a list it reads, a flow that triggers on changes, a function that re-indexes on demand. Six steps.

01 Scaffold the extension

```
yo @microsoft/sharepoint
  --component-type extension
```

Out:

- ApplicationCustomizer + ListViewCommandSet projects
- configured for Node 22 and SPFx 1.21

02 Header banner reads config

```
sp.web.lists.getByTitle(
  "Banners").items.filter(`Active eq 1`);
```

Out:

- Banner mounted in *PlaceholderTop*
- *localStorage* for dismiss tracking

03 Deploy with App Catalog

```
gulp bundle --ship
 && gulp package-solution --ship
```

Out:

- Tenant-deployed *.sppkg*
- Banner visible on every modern page

04 Register & admin-consent

```
Sites.Selected + User.ReadBasic.All
  in package-solution.json
```

Out:

- Admin approves once in Entra
- Failure path tested with a fresh tenant

05 Approval flow triggered

```
When an item is modified
  → Start and wait for approval
```

Out:

- Approval runs in Teams inbox
- HTTP step writes back to Banners list

06 Function re-indexes on demand

```
POST /api/reindex listId
 · Graph /search/index
```

Out:

- Function logs to Application Insights
- Search reflects update within minutes

Acceptance criteria

Banner shows on every modern page until acknowledged locally; tenant admin approves once; approval adjusts banner; function confirms re-index; search finds new banner within minutes.